

Chapter 2: B-Tree Basics - The Backbone of Database Indexes

Table of Contents

1. Why B-Trees?
 2. From Binary Search Trees to B-Trees
 3. B-Tree Properties and Invariants
 4. B+Tree: The Database Variant
 5. Node Structure and Layout
 6. Lookup Operations
 7. Insertion Operations
 8. Deletion Operations
 9. Balancing Mechanics
 10. Disk I/O Optimization
-

Why B-Trees?

B-Trees are the **dominant data structure** for database indexes. Understanding why requires examining the fundamental problem: efficient disk access.

THE DISK ACCESS PROBLEM

SCENARIO: Search for key 42 among 1 million records

APPROACH 1: Linear Scan

[1][2][3][4]...[42]...[1000000]

Average: 500,000 disk reads

If each read = 10ms → 5000 seconds = 83 minutes!

APPROACH 2: Binary Search (sorted array)

$\log_2(1,000,000) \approx 20$ comparisons

BUT: Each comparison = 1 disk seek

20 seeks × 10ms = 200ms

APPROACH 3: B-Tree (branching factor = 100)

$\log_{100}(1,000,000) \approx 3$ levels

Only 3 disk reads!

3 reads × 10ms = 30ms

KEY INSIGHT: Maximize data read per disk access

The High Branching Factor Advantage

TREE HEIGHT COMPARISON

For $N = 1,000,000,000$ (1 billion) records:

BINARY TREE (branching factor = 2)

Height = $\log_2(10^9) \approx 30$ levels
→ 30 disk I/O operations per lookup

B-TREE (branching factor = 200)

Height = $\log_{200}(10^9) \approx 4$ levels
→ 4 disk I/O operations per lookup

VISUAL COMPARISON

Binary Tree: [root]

```
      / \
     [2] [3]
    /\  /\
   ... ..
```

Height grows quickly
(30 levels for 1B records)

B-Tree: [root: contains 200 keys and 201 pointers]

```
      /  |  |  |  \
    [200][200][200]...[200]
     / | \
    [....]
```

Level 1: 201 children
Level 2: 40,401 children
Level 3: 8,120,601 children
Level 4: 1.6 billion entries!

From Binary Search Trees to B-Trees

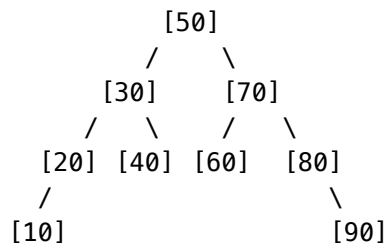
Binary Search Tree (BST) Review

BINARY SEARCH TREE

PROPERTIES

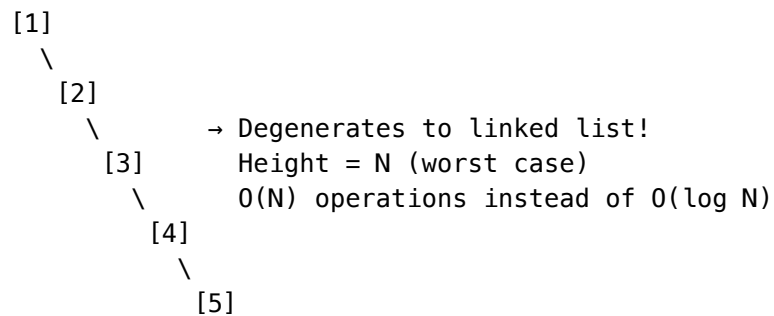
- Each node contains ONE key
- Left subtree: all keys < node key
- Right subtree: all keys > node key

EXAMPLE BST



PROBLEM: Unbalanced trees

Insert: 1, 2, 3, 4, 5 (sorted order)



Self-Balancing BSTs (AVL, Red-Black)

SELF-BALANCING BSTs

AVL TREE

- Balance factor = $\text{height}(\text{left}) - \text{height}(\text{right})$
- Invariant: $|\text{balance factor}| \leq 1$ for every node
- Rotations to rebalance after insert/delete

RED-BLACK TREE

- Each node is Red or Black
- Root is always Black
- No two Red nodes in a row (parent-child)
- Every path from root to NULL has same # of Black nodes

WHY NOT USE THESE FOR DATABASES?

Problem 1: Low branching factor

- Still binary (2 children per node)
- $\log_2(N)$ height $\rightarrow$ too many disk reads

Problem 2: Each node = 1 disk page?

- If page = 4KB and key = 8 bytes
- Utilization = $8 / 4096 = 0.2\%$ (horrible!)

Problem 3: Rebalancing locality

- Rotations may access distant nodes
- Many disk seeks for single operation

B-Tree Properties and Invariants

B-TREE FORMAL DEFINITION

A B-Tree of order M (or degree M) satisfies:

INVARIANT 1: Node Key Limits

- Root: 1 to $M-1$ keys
- Internal nodes: $\lceil M/2 \rceil - 1$ to $M-1$ keys
- Leaf nodes: $\lceil M/2 \rceil - 1$ to $M-1$ keys

INVARIANT 2: Child Pointer Count

- If a node has K keys, it has $K+1$ children (for internal nodes)
- Leaf nodes have no children

INVARIANT 3: All Leaves at Same Depth

- Tree is perfectly balanced
- Every path from root to leaf has same length

INVARIANT 4: Keys are Sorted Within Node

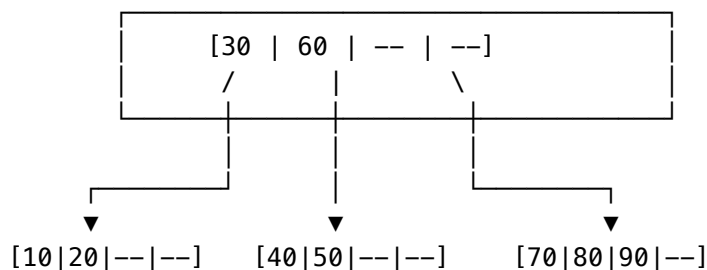
- $K_1 < K_2 < K_3 < \dots < K_n$

INVARIANT 5: Subtree Ordering

- All keys in subtree pointed by P_i are:
 - Greater than K_{i-1} (if $i > 0$)
 - Less than K_i (if $i < \text{number of keys}$)

EXAMPLE: B-Tree of Order 5 ($M=5$)

- Each node: 2 to 4 keys ($\lceil 5/2 \rceil - 1 = 2$, $M-1 = 4$)
- Each internal node: 3 to 5 children

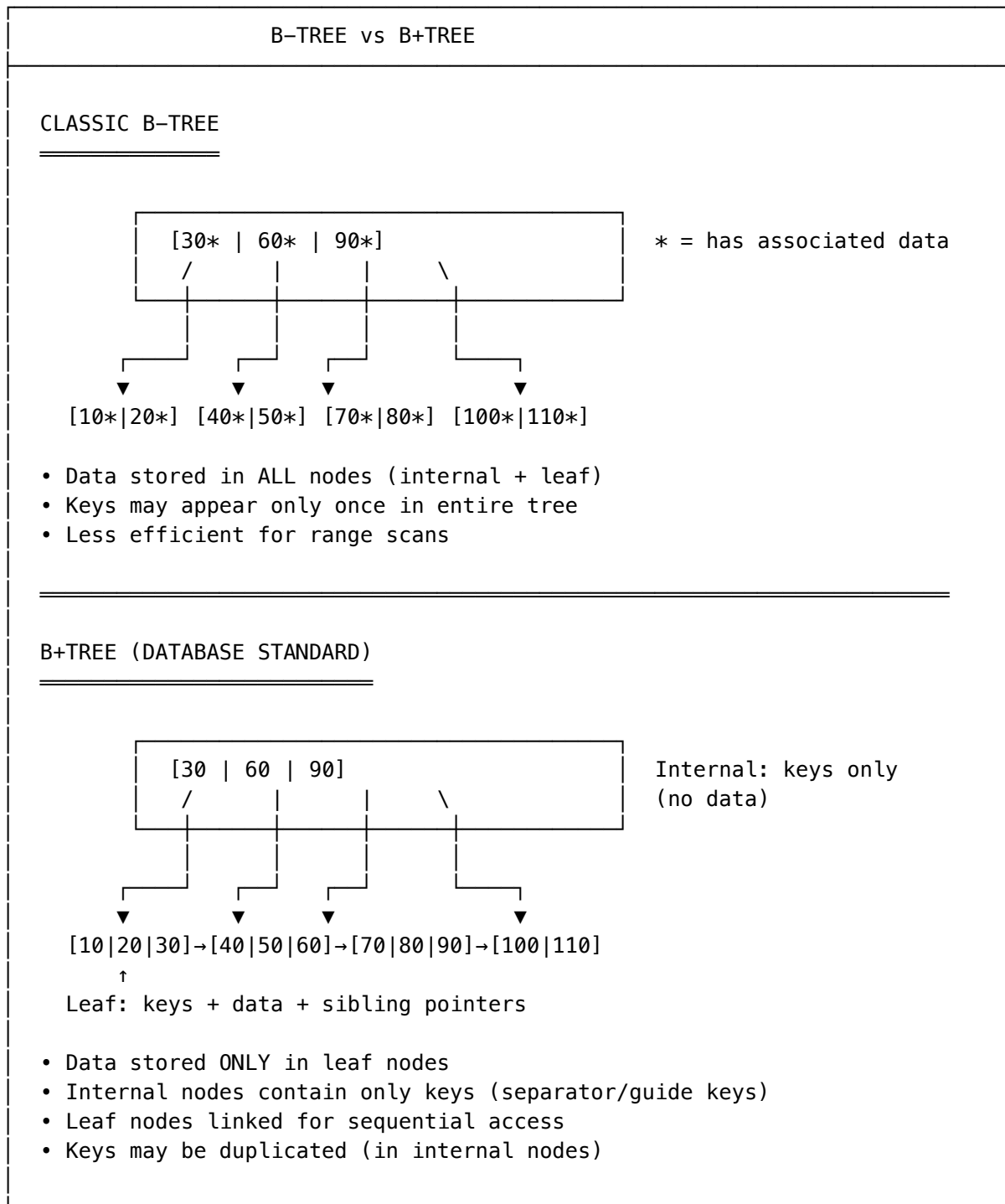


Why These Invariants Matter

INVARIANT BENEFITS
BENEFIT 1: Guaranteed Balance (Invariant 3)
• Height = $O(\log_M N)$ • Predictable performance • No worst-case linear degradation
BENEFIT 2: High Fanout (Invariants 1, 2)
• Each node holds many keys $\rightarrow$ low height • Each disk read yields many comparisons • Fewer I/O operations per lookup
BENEFIT 3: Space Efficiency (Invariant 1)
• Minimum fill factor: 50% • Nodes always at least half full • Prevents sparse trees wasting disk space
BENEFIT 4: Sorted Access (Invariants 4, 5)
• In-order traversal yields sorted data • Efficient range queries • Binary search within nodes

B+Tree: The Database Variant

Most databases use B+Trees rather than classic B-Trees. Here's why:



B+Tree Advantages for Databases

WHY DATABASES USE B+TREES

ADVANTAGE 1: Higher Fanout in Internal Nodes

B-Tree internal node: $[Key_1 | Data_1 | Ptr_1 | Key_2 | Data_2 | Ptr_2 | \dots]$

B+Tree internal node: $[Key_1 | Ptr_1 | Key_2 | Ptr_2 | Key_3 | Ptr_3 | \dots]$

If page = 8KB, key = 8 bytes, data = 100 bytes, pointer = 8 bytes:

- B-Tree: $8192 / (8 + 100 + 8) \approx 70$ keys per node
- B+Tree: $8192 / (8 + 8) \approx 500$ keys per node

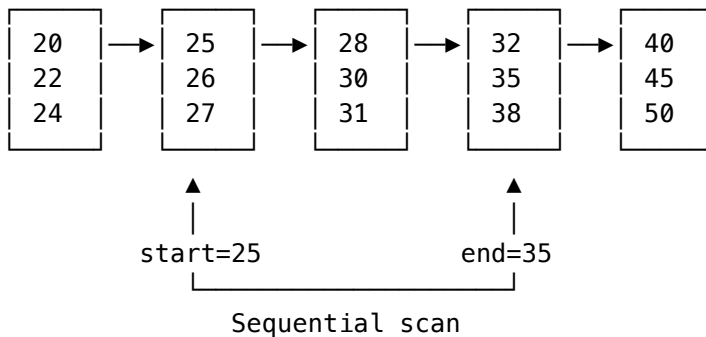
→ B+Tree has 7x higher fanout → fewer levels → fewer disk reads

ADVANTAGE 2: Efficient Range Scans

Query: `SELECT * FROM users WHERE age BETWEEN 25 AND 35`

B+Tree approach:

1. Find leaf with age=25 (log N lookups)
2. Follow sibling pointers: $[25] \rightarrow [26] \rightarrow [27] \rightarrow \dots \rightarrow [35]$
3. Sequential scan through linked leaves



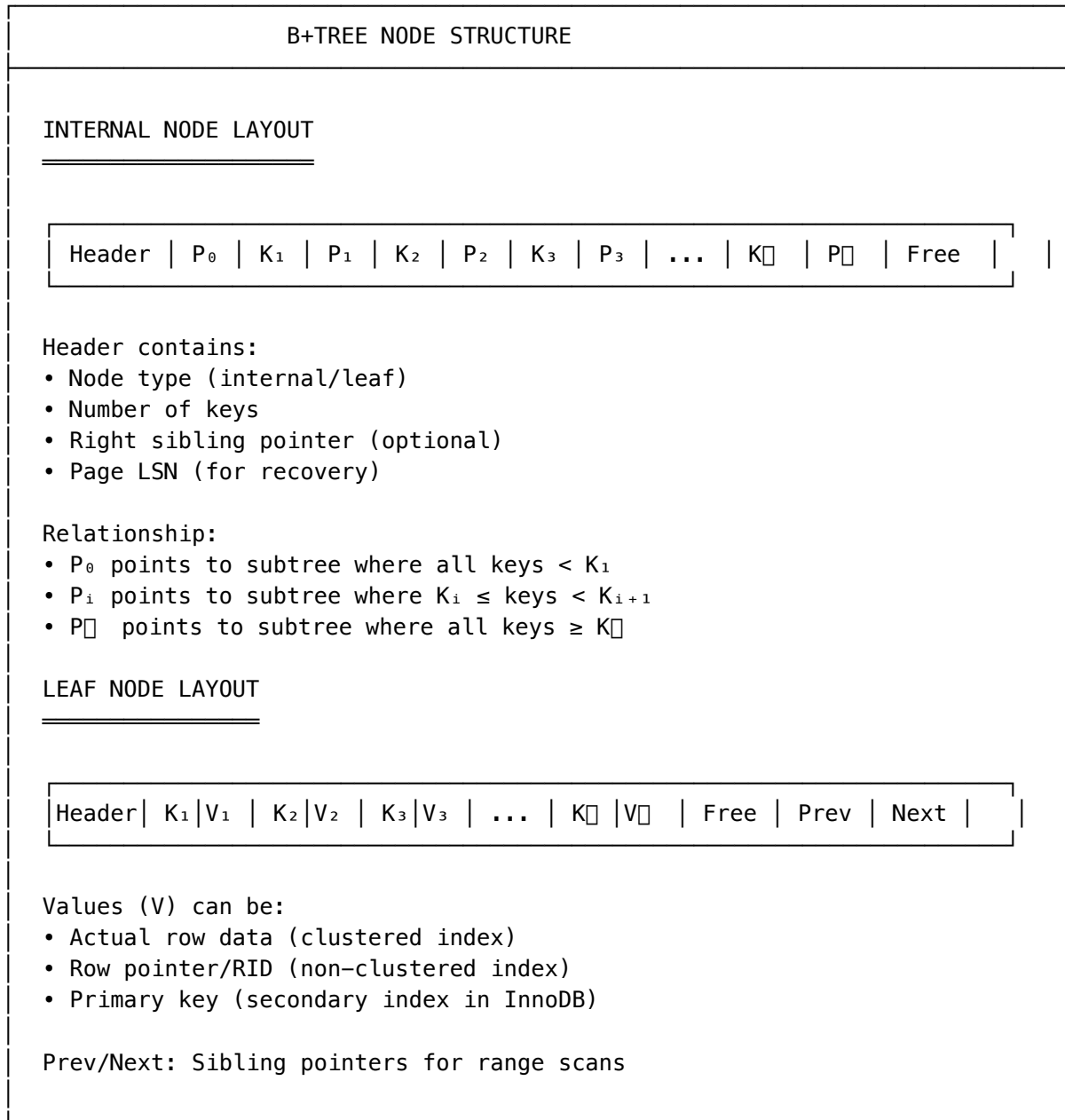
ADVANTAGE 3: Consistent Performance

- All data at same depth (leaf level)
- Every lookup traverses same number of levels
- Predictable latency

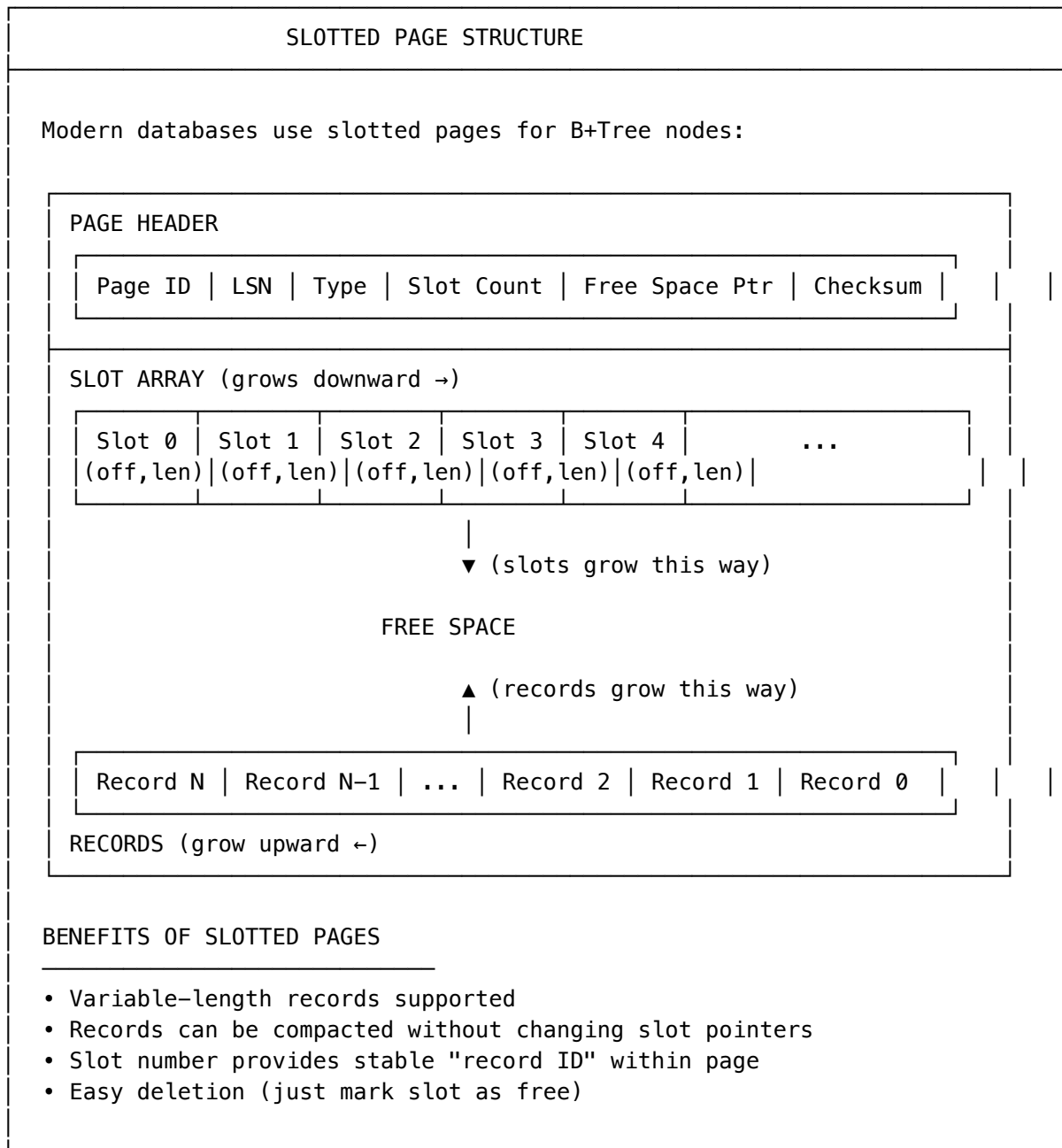
ADVANTAGE 4: Simpler Caching Strategy

- Internal nodes are smaller → more fit in cache
- Cache internal nodes (hot), fetch leaves on demand
- Better memory utilization

Node Structure and Layout



Slotted Page Organization



Lookup Operations

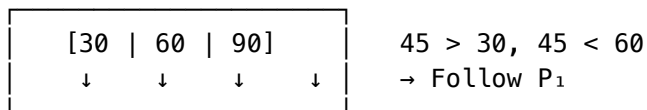
B+TREE POINT LOOKUP ALGORITHM

ALGORITHM: Search(key)

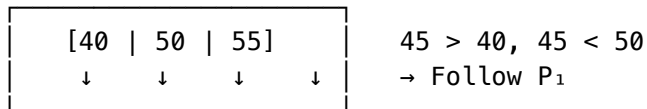
1. Start at root node
2. While current node is internal:
 - a. Binary search for key position
 - b. Follow appropriate child pointer
3. At leaf node:
 - a. Binary search for key
 - b. Return value if found, NULL otherwise

EXAMPLE: Search for key 45

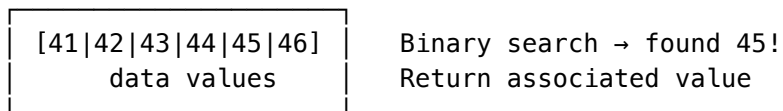
Step 1: Start at root



Step 2: Internal node



Step 3: Leaf node (found!)



COMPLEXITY

- Tree traversal: $O(\log_M N)$ where M = branching factor
- Binary search within node: $O(\log M)$
- Total: $O(\log_M N \times \log M) = O(\log N)$
- Disk I/O: $O(\log_M N)$ pages read

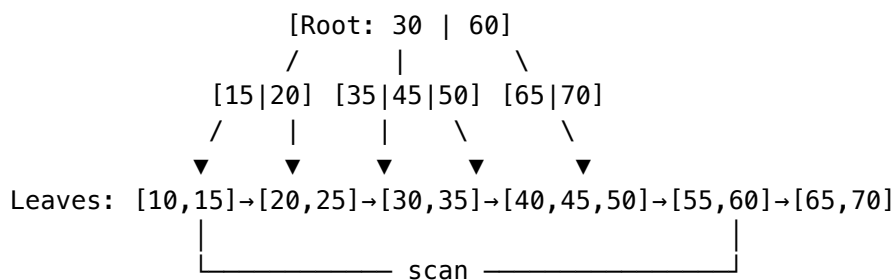
Range Query Algorithm

B+TREE RANGE SCAN ALGORITHM

ALGORITHM: RangeScan(low_key, high_key)

1. Search for low_key → find starting leaf
2. Scan leaf for keys $\geq$ low_key
3. While current key $\leq$ high_key:
 - a. Collect matching records
 - b. When leaf exhausted, follow "next" pointer
4. Return collected records

EXAMPLE: Range [25, 55]



1. Search(25) → land on leaf [20,25]
2. Scan: 25 ✓
3. Next leaf [30,35]: 30 ✓, 35 ✓
4. Next leaf [40,45,50]: 40 ✓, 45 ✓, 50 ✓
5. Next leaf [55,60]: 55 ✓, 60 > 55 → stop
6. Return: {25, 30, 35, 40, 45, 50, 55}

PERFORMANCE

- Initial lookup: $O(\log N)$
- Scan: $O(K)$ where K = number of results
- Sequential I/O (leaves are linked) → very efficient

Insertion Operations

B+TREE INSERTION ALGORITHM

ALGORITHM: Insert(key, value)

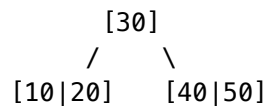
1. Search for appropriate leaf node
2. If leaf has space:
 - a. Insert key-value pair in sorted order
 - b. Done!
3. If leaf is full (overflow):
 - a. Split leaf into two nodes
 - b. Distribute keys evenly
 - c. Insert middle key into parent
 - d. If parent overflows, split recursively

Insertion Without Split

SIMPLE INSERTION (NO SPLIT)

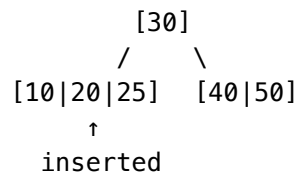
Insert key 25 into B+Tree (order 4: max 3 keys per node)

BEFORE:



- Step 1: Search for 25 → leaf [10|20]
Step 2: Leaf has space (2 keys, max 3)
Step 3: Insert 25 in sorted position

AFTER:



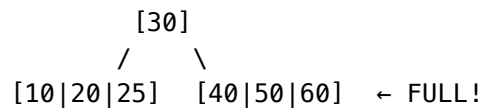
No structural changes needed!

Leaf Node Split

LEAF SPLIT INSERTION

Insert key 35 into B+Tree (order 4: max 3 keys per node)

BEFORE:



Step 1: Search for 35 → leaf [40|50|60]

Step 2: Leaf is FULL (3 keys, max 3)

Step 3: Split required!

SPLIT PROCESS

1. Insert key logically: [35|40|50|60] (4 keys – overflow!)

2. Split at midpoint:

Left: [35|40]

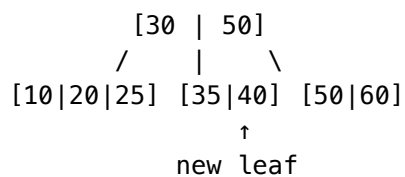
Right: [50|60]

↑ Copy 50 up to parent

3. Link siblings:

[35|40] ↔ [50|60]

AFTER:



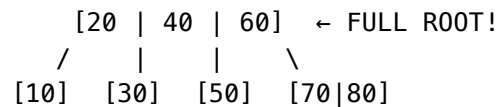
Note: In B+Trees, the split key (50) is COPIED up, not moved
(50 remains in the right leaf for completeness)

Internal Node Split (Cascading Split)

INTERNAL NODE SPLIT

When a leaf split requires inserting into a full parent:

BEFORE (root is full):



Insert 75: causes split of [70|80] → need to insert into root

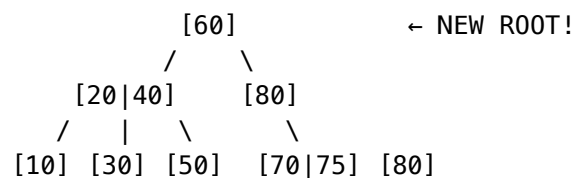
SPLIT PROCESS

1. Leaf [70|80] + 75 → [70|75] and [80]
Push 80 up to parent
2. Parent [20|40|60] + 80 → overflow!
[20|40|60|80] needs split
3. Split parent at midpoint:
Left: [20|40]
Right: [60|80]
Push 50 up (middle key – but wait, we need to recalculate)

Actually: [20|40|60|80] splits as:

Left: [20|40]
Right: [80]
Middle key 60 pushed UP to new root

AFTER:



KEY INSIGHT: In B+Trees, when internal nodes split:

- Middle key is **MOVED** up (not copied)
- This differs from leaf splits where key is **COPIED**

Deletion Operations

B+TREE DELETION ALGORITHM

ALGORITHM: Delete(key)

1. Search for key in leaf node
2. Delete key from leaf
3. If leaf still meets minimum occupancy:
 - a. Done! (update parent key if needed)
4. If leaf underflows (below minimum):
 - a. Try borrowing from sibling
 - b. If cannot borrow, merge with sibling
 - c. Update/delete parent key
 - d. If parent underflows, recurse upward

MINIMUM OCCUPANCY

Order M B+Tree:

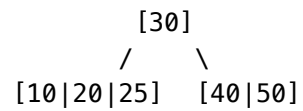
- Minimum keys in leaf: $\lceil M/2 \rceil - 1$
- Minimum keys in internal: $\lceil M/2 \rceil - 1$
- Root: no minimum (can have 1 key)

Deletion Without Underflow

SIMPLE DELETION (NO UNDERFLOW)

Delete key 25 from B+Tree (order 5: min 2 keys per node)

BEFORE:

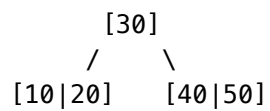


Step 1: Search for 25 → leaf [10|20|25]

Step 2: Delete 25

Step 3: Leaf has 2 keys (minimum met)

AFTER:



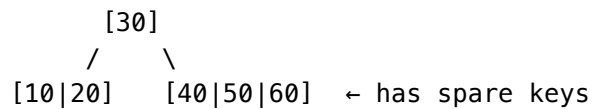
No rebalancing needed!

Redistribution (Borrowing from Sibling)

REDISTRIBUTION / BORROWING

When deletion causes underflow but sibling has spare keys:

BEFORE: Delete 10 (order 5: min 2 keys per node)



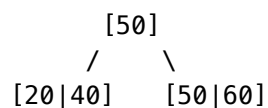
Problem: After deleting 10, left leaf has only 1 key (underflow!)
[20] – only 1 key, minimum is 2

Solution: Borrow from right sibling

REDISTRIBUTION PROCESS

1. Delete 10 → left leaf becomes [20] (underflow)
2. Right sibling [40|50|60] has 3 keys (can spare)
3. Borrow: Move 40 from right sibling to left leaf
4. Update parent separator: 30 → 50

AFTER:



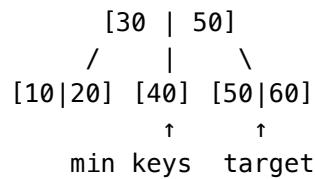
Both leaves now have 2 keys (minimum satisfied)

Merge (Coalescence)

NODE MERGE / COALESCENCE

When neither sibling can spare keys:

BEFORE: Delete 50 (order 5: min 2 keys per node)



Problem: Deleting 50 leaves [60] with 1 key (underflow)

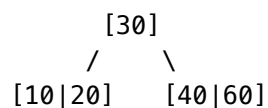
Left sibling [40] has minimum keys – can't borrow

Solution: Merge [40] and [60]

MERGE PROCESS

1. Delete 50 → [60] underflows
2. Merge [40] and [60] → [40|60]
3. Remove separator key 50 from parent

AFTER:



Parent now has 1 key (still valid for root)

CASCADE: If parent underflows, merge/borrow recursively
If root becomes empty, child becomes new root

Balancing Mechanics

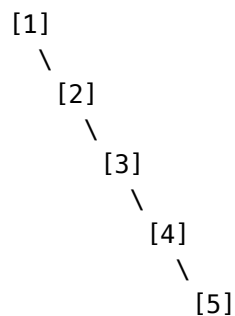
B+TREE BALANCE GUARANTEE

WHY B+TREES ARE ALWAYS BALANCED

1. Growth happens at the ROOT (splits propagate UP)
 - Unlike BSTs where growth happens at leaves
 - All leaves always at same depth
2. Shrinkage happens at the ROOT (merges propagate UP)
 - When root has no keys, tree height decreases
 - All leaves still at same depth

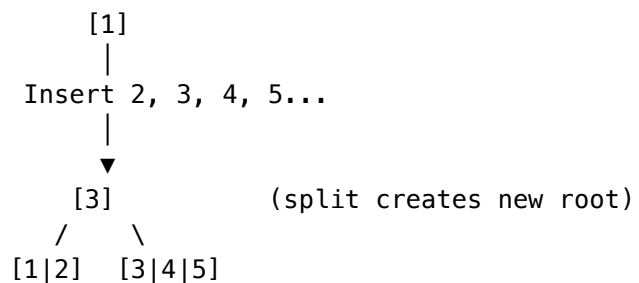
VISUALIZATION

BST Growth (unbalanced):



Height: 5 (degenerates)

B+Tree Growth (balanced):



Height: 2 (balanced!)

Space Utilization Analysis

SPACE UTILIZATION

FILL FACTOR ANALYSIS

For B+Tree of order M:

- Maximum keys per node: $M - 1$
- Minimum keys per node: $\lceil M/2 \rceil - 1$

Minimum fill factor = $(\lceil M/2 \rceil - 1) / (M - 1) \approx 50\%$

Example: Order 100 B+Tree

- Max keys: 99
- Min keys: 49
- Min fill: $49/99 = 49.5\%$

WORST CASE vs AVERAGE CASE

Worst case: All nodes at minimum (50% full)

Average case (random inserts): ~69% full

Best case (bulk load): ~100% full (with proper algorithms)

IMPLICATIONS

- Even worst case guarantees reasonable space usage
- No need for periodic rebalancing/compaction
- Sequential inserts may cause suboptimal fill
- Bulk loading can achieve near-100% utilization

Disk I/O Optimization

B+TREE I/O OPTIMIZATION TECHNIQUES

1. PAGE SIZE ALIGNMENT

- Node size = disk page size (typically 4KB, 8KB, or 16KB)
- One I/O = one complete node read
- Maximizes data retrieved per disk access

Typical page sizes:

- PostgreSQL: 8 KB
- MySQL InnoDB: 16 KB
- SQLite: 4 KB (configurable)
- Oracle: 8 KB (configurable)

2. FANOUT MAXIMIZATION

- More keys per node = lower tree height = fewer I/Os
- Key compression (prefix compression, suffix truncation)
- Pointer compression (relative page numbers)

Example: 16KB page, 8-byte keys, 8-byte pointers

Fanout = $16384 / (8 + 8) \approx 1000$ children per node!

Height for 1 billion records:

$\log_{1000}(10^9) = 3$ levels $\rightarrow$ only 3 I/Os per lookup

3. BUFFER POOL CACHING

- Keep hot pages (especially root and upper levels) in memory
- Root node: almost always cached (1 page)
- Level 1: likely cached (few hundred pages)
- Leaves: may require disk I/O

Effective I/O per lookup $\approx 1-2$ (not tree height)

4. PREFETCHING

- For range scans, prefetch next leaf pages
- Sequential read is faster than random read
- Exploit disk read-ahead capabilities

5. BULK LOADING

- For initial data load, build bottom-up
- Sort data first, then create leaves
- Build internal nodes from leaf boundaries
- Achieves $\sim 100\%$ fill factor

Practical Performance Numbers

REAL-WORLD B+TREE PERFORMANCE

LOOKUP LATENCY (assuming warm cache for upper levels)

Records	Tree Height	Uncached I/Os	Typical Latency
10K	2	1	~0.1 ms (SSD)
1M	3	1-2	~0.2 ms (SSD)
100M	3-4	1-2	~0.3 ms (SSD)
1B	4	1-2	~0.4 ms (SSD)

With spinning disk: multiply by 10-100x

RANGE SCAN THROUGHPUT

Sequential leaf scan: ~100-500 MB/s (limited by disk/network)

Records per second: millions (depends on record size)

INSERT THROUGHPUT

Random inserts: 10K-50K ops/sec (depends on write strategy)

Bulk load: 100K-1M+ records/sec

KEY INSIGHT

B+Trees are optimized for disk access patterns:

- Read a page → process many keys
- Sequential access → exploit disk prefetching
- Cache upper levels → minimize random I/O

Summary

CHAPTER 2 KEY TAKEAWAYS

1. B-Trees exist because disk I/O is expensive
 - High branching factor minimizes tree height
 - Each disk read yields maximum useful information
2. B+Trees are the database standard
 - Data only in leaves (higher fanout for internal nodes)
 - Linked leaves enable efficient range scans
 - Consistent lookup performance (all data at same depth)
3. Operations maintain balance automatically
 - Inserts: split nodes when full, propagate upward
 - Deletes: borrow or merge when underflow, propagate upward
 - Tree grows/shrinks at the root, not leaves
4. Performance characteristics
 - $O(\log N)$ for all operations
 - In practice: 3-4 I/Os for billions of records
 - With caching: often just 1-2 I/Os

WHAT'S NEXT

Chapter 3: B-Tree Variants

- B*-Trees (higher fill factor)
- B-link Trees (better concurrency)
- Copy-on-Write B-Trees (MVCC support)
- Fractal Trees (write optimization)